1 测试理论笔记

1 软件测试概述

什么是软件

- 硬件概念: 由电子, 机械和光电元件等等组成的各种物理装置的总称

> 特点: 看得见摸得着

- 软件概念: 是一系列按照特定顺序组织的计算机数据和指令的集合

> 特点: 看得见摸不着

软件分类

- 按开发规模划分
  - 小型: 开发周期4个月内 -- 参与人数10人以下
  - 中型: 开发周期1年内 -- 参与人数10~100人
  - 大型: 开发周期1年以上 -- 参与人数100人以上
- 按用户对象划分
  - 产品: 微信/office/qq/百度
  - 项目: 为企业定制的OA系统
- 按是否需要联网划分
  - 单机: 蜘蛛纸牌/画图工具
  - 网络:
    - C/S架构: qq/LOL
      - B/S架构: 搜狐/百度
- 按功能划分
  - 应用软件: 淘宝/有道词典
  - 系统软件: 操作系统/驱动软件

什么是软件测试

- 概念: 在规定条件下对程序进行操作, 以发现程序错误, 并对其是否能满足设计要求进行评估的过程
- 目的: 找缺陷 (找bug)

- 对象:
  - 源程序
  - 目标程序
  - 数据
  - 文档

2 软件开发流程

2.1 瀑布模型

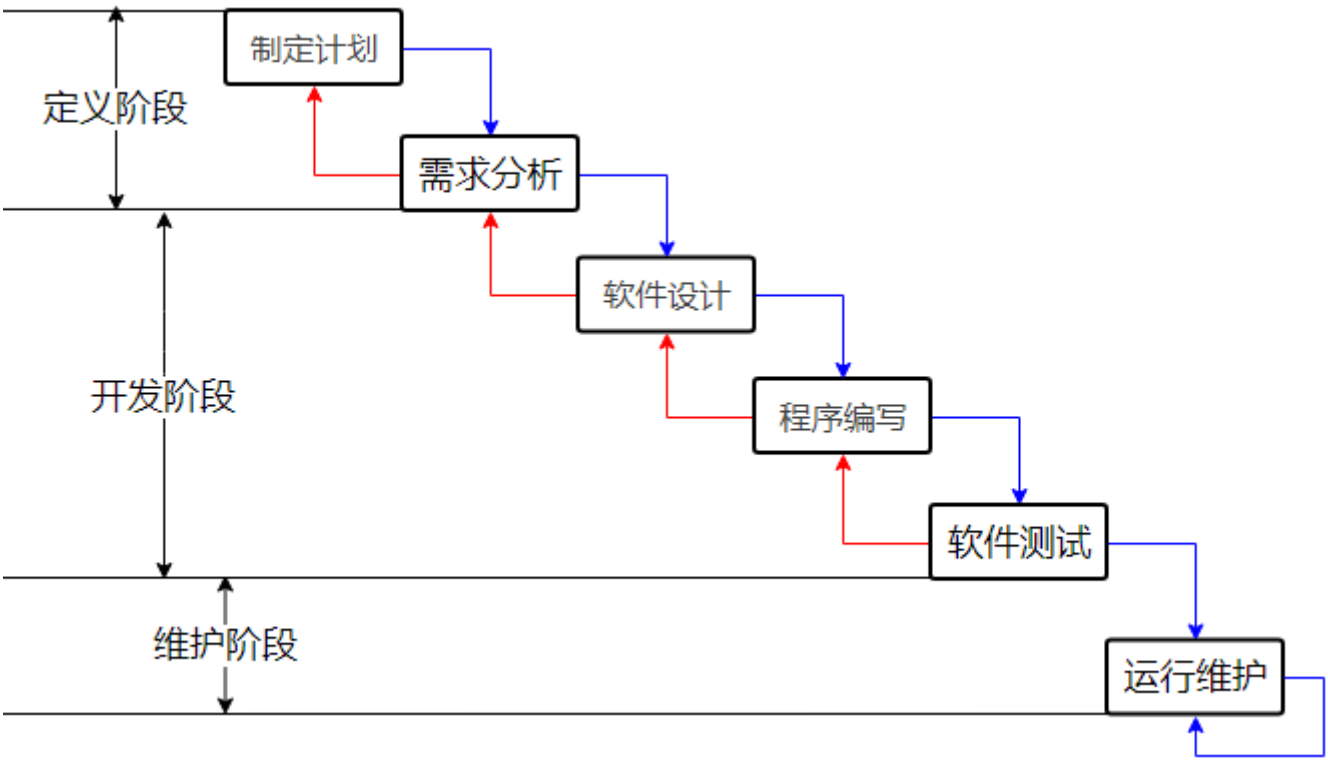
开发模型分类

- 边做边改模型
- 瀑布模型
- 快速原型模型
- 增量模型
- 螺旋模型
- 喷泉模型
- ...

概念

将软件生命周期划分为制定计划, 需求分析, 软件设计, 程序编写, 软件测试 和 运行维护六个基本活动, 并且规定了它们自上而下, 相互衔接的固定次序如同瀑布流水, 逐级下落

图示



> 组成阶段: 6个**角色和产出**

	制定计划	需求分析	软件设计		程序编写	软件测试	运行维护
			概要设计	详细设计			
参与角色	boss, 产品经理	产品经理, 开发	开发(老大)	开发	开发	测试	运维
产出	可行性分析报告 项目计划书	**需求文档**(PRD) **[是一切测试活动的依据]**	概要设计说明书	详细设计说明书	程序	测试报告 操作手册	运维手册

小结

- 特点: 线性模型, 是其他模型的基础
- 应用场景: 需求清晰的大型项目, 如: 建筑, 银行, 保险等
- 优点: 阶段清楚
- 缺点: 返工量大, 不适用于需求变化的项目

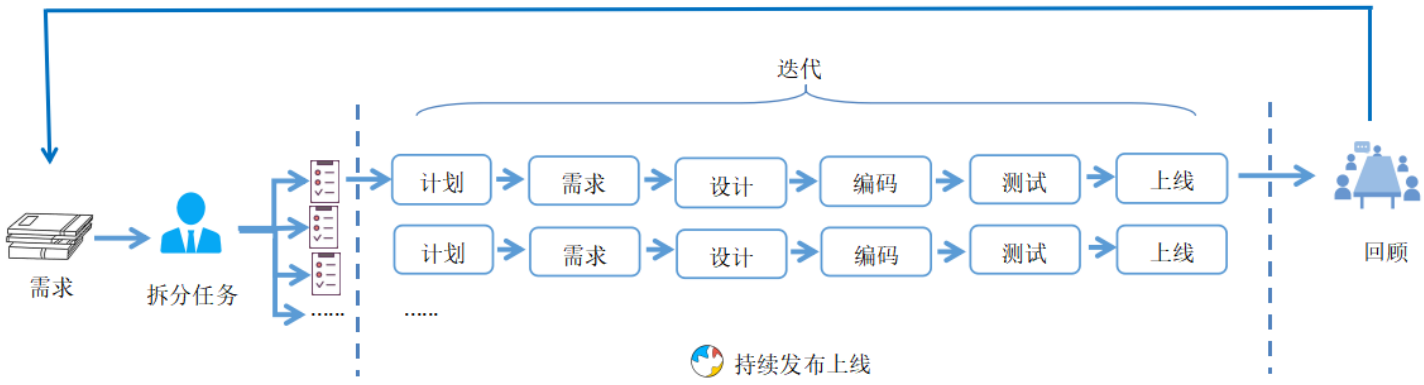
面试题

软件的生命周期?[1]

立项 -> 计划 -> 需求分析 -> 设计 -> 编码 -> 测试 -> 运行维护

2.2 敏捷开发

90%以上的公司采用敏捷开发



背景

- 是一种20世纪90年代后逐渐引起广泛关注的新型软件开发方法
  - 也叫 敏捷方法 敏捷过程 敏捷模型
- 它的具体名称, 理念, 过程, 术语都不尽相同
- 它是一种以人为本, 迭代, 循序渐进的开发方法
-

图示

基本原则

- 以人为本: 相信自己的团队, 规律的反省/优化, 重视客户胜过合同
- 一切从简: 简化流程, 文档, 交流方式等
- 迭代: 持续交付, 周期越短越好

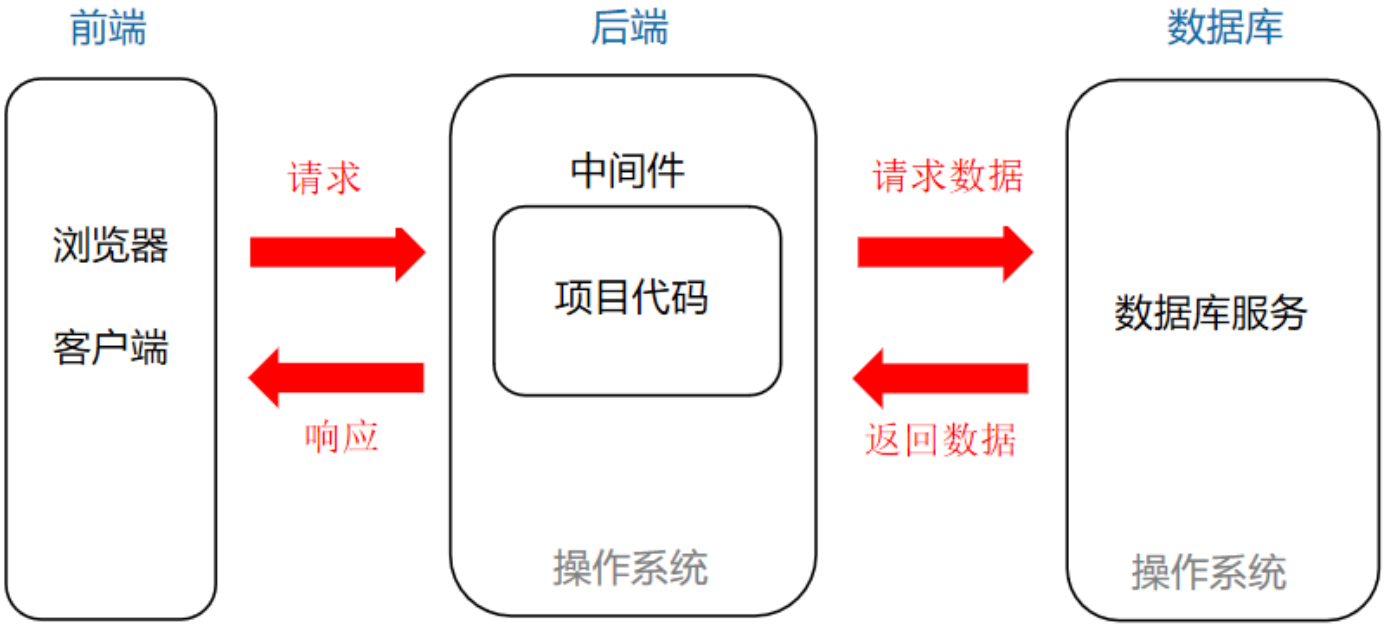
2.3 项目构成[补充]

说明

一个项目基本都是由**前端**, **后端** 和 **数据库** 三个部分构成

- 前端: 用户所使用的**浏览器**或**客户端**
- 后端: 运行在**应用服务器**上, 作为前端和数据库的中间人, 处理业务和数据
- 数据库: 运行在**数据服务器**上, 用于存储数据

图示



中间件概念

介于系统和应用之间的一类软件

> 可以简单理解为, 中间件可以在系统上帮助应用开放端口

常用的中间件

- Tomcat: 主要运行 Java 程序
- Apache: 主要运行 PHP 程序

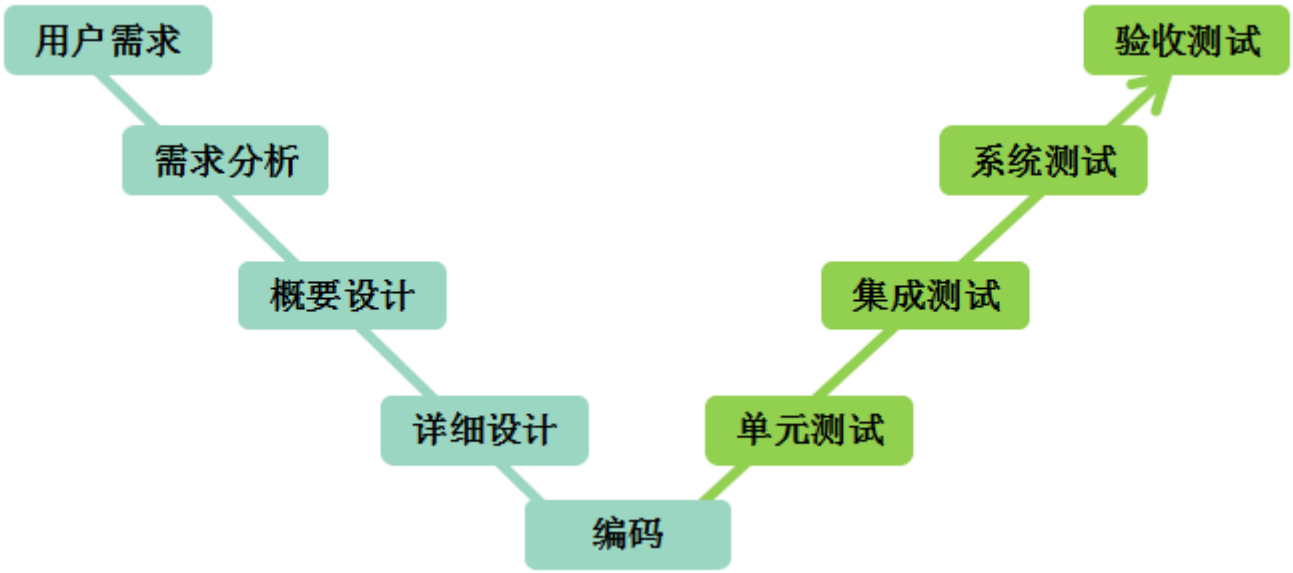
3 软件测试流程

软件测试模型分类

- V模型 (重点)
- W模型 (重点)
- X模型
- H模型

V模型

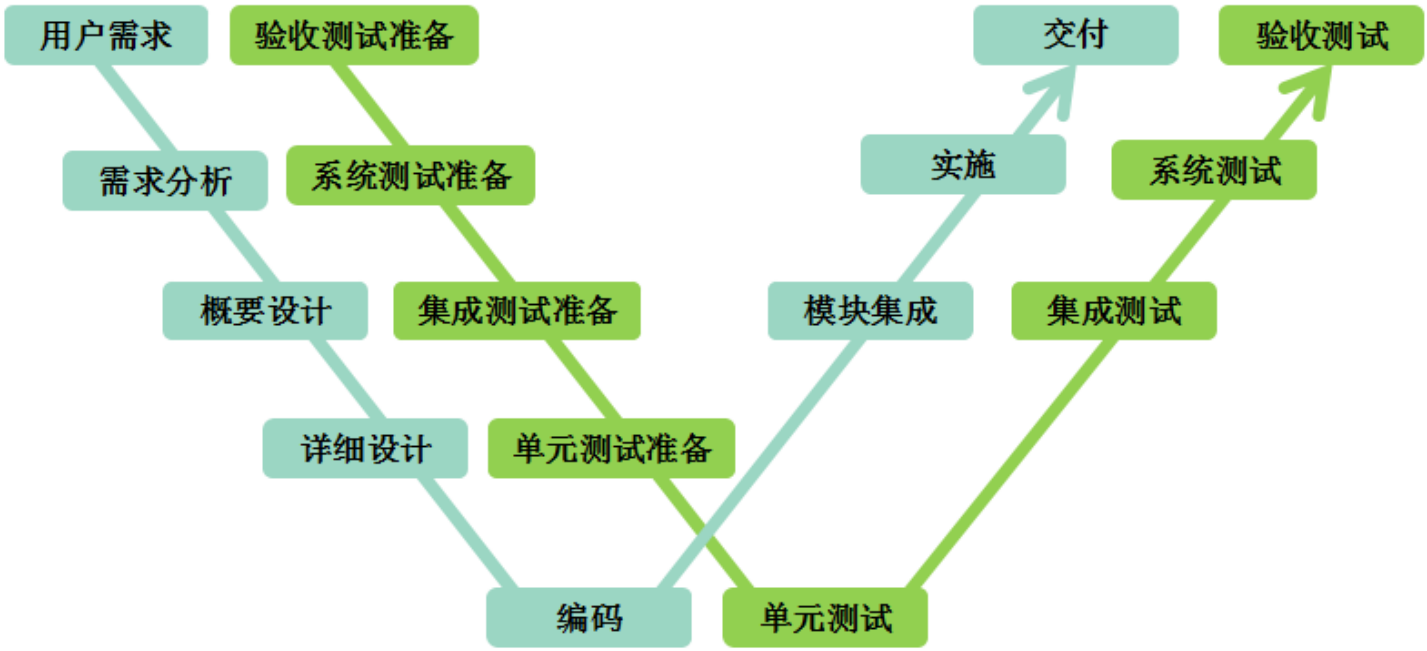
- 概念: 以"编码"为黄金分割线, 把整个过程分为开发和测试, 并且开发和测试之间是串行的关系
- 图示



- 特点: 开发与测试串行, 测试介入过晚, 返工量大

W模型

- 概念: W模型是由两个V模型组成, 一个是开发阶段, 一个是测试阶段, 又称为双V模型
- 图示



- 特点: 开发与测试并行, 测试可以提早介入, 但是总体流程依然是串行, 不支持迭代

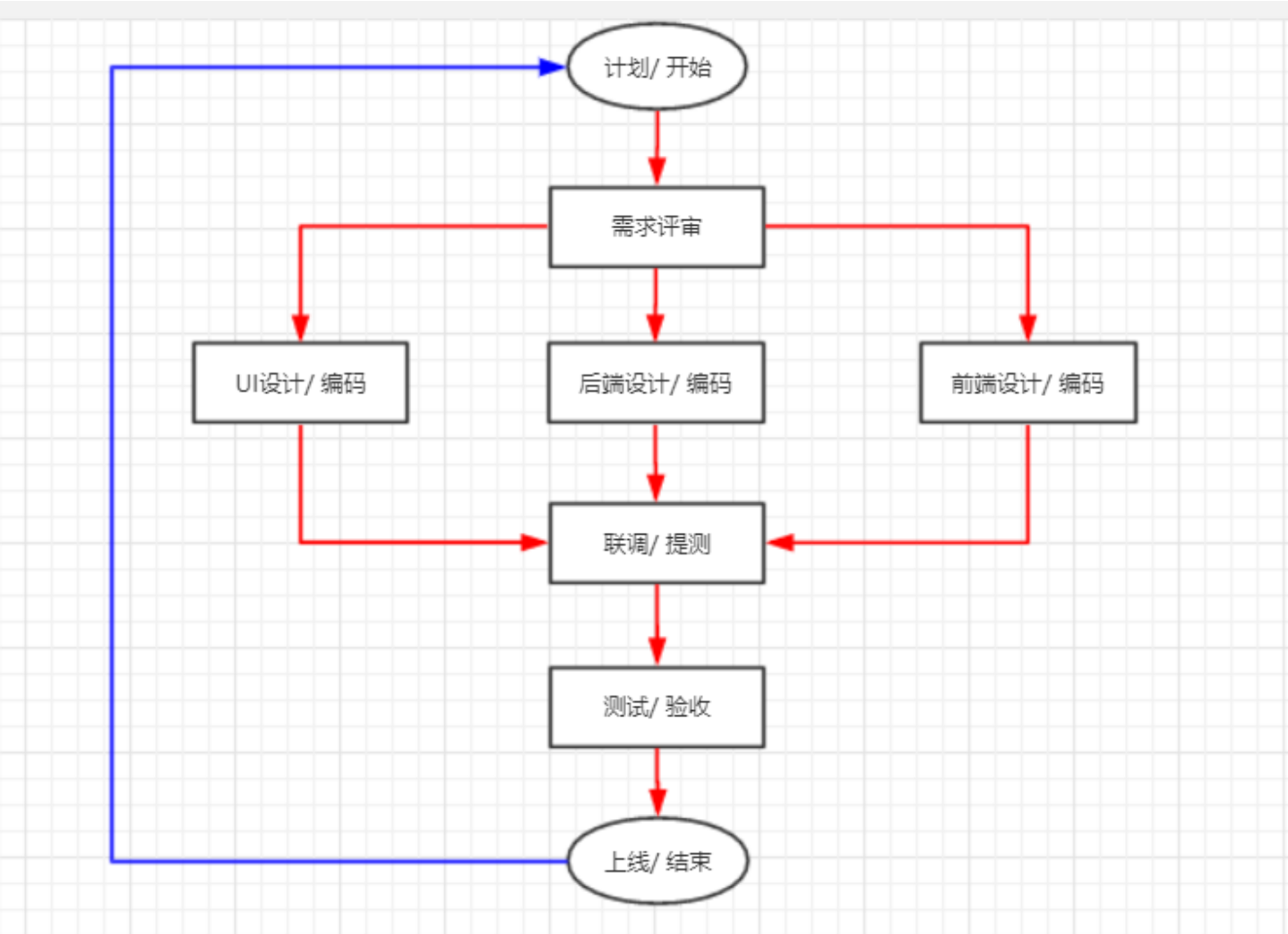
4 最佳实践流程-1

团队角色介绍

- 产品经理/项目经理 (负责人)

- UI
- 前端开发
- 后端开发
- 测试
- 运维/运营

图示



5 软件测试分类

5.1 按阶段划分

单元测试的概念

指对软件中的最小可测单元 (一般可以理解为是函数) 进行检查和验证

集成测试的概念

在单元测试的基础上, 将所有模块按照设计要求组装成子系统或系统, 进行测试

系统测试的概念

系统测试是将 **软件, 操作系统** 和 **硬件** 看作一个整体, 在实际运行环境下进行测试

验收测试的概念

对完成的系统是否满足 **最开始的需求** 进行验证

> 分类
>
> - 按用户对象划分
> - 项目验收: **甲方**发起, 验证**乙方**系统是否满足甲方**业务需求**
> - 产品验收: **产品经理**发起, 验证自研软件是否满足**用户需求**
> - 按阶段划分
> - α测试 (内测): 只有公司内部成员参与
> - β测试 (公测): 由用户完成

测试阶段 -- 负责人员 -- 项目环境 的关系

单元 -- 开发人员 -- 开发环境

集成 -- 开发人员 -- 开发环境

系统 -- 测试人员 -- 测试环境

验收 -- 产品经理/甲方 -- 预生产环境

主要的项目环境

- 开发环境
- 测试环境
- 预生产环境
- 线上环境/生产环境

5.2 按是否考虑代码逻辑划分

黑盒测试

- 概念: 把测试对象看作一个黑盒子, 测试时, 测试人员不用考虑盒子里的逻辑结构, 只需检查程序的功能是否符合需求文档
- 重点: 验证程序的功能是否符合需求文档
- 测试依据: 需求文档
- 分类
  - 界面测试/UI测试: 检查页面元素是否符合UI设计、页面是否美观

- 功能测试: 检查产品功能是否满足需求
- 易用性测试: 用户体验
- 兼容性测试: 验证软件在各个环境下功能是否正常
- 性能测试: 模拟用户场景, 测试系统的各项性能指标, 查看是否满足需求
- ...

白盒测试

- 概念: 与黑盒相反, 这种方法是把测试对象看作一个透明的盒子, 测试时, 测试人员会对程序的所有逻辑路径进行测试, 检验每条路径是否都能走通
- 重点: 验证源代码的逻辑结构是否符合设计
- 测试依据: 代码规范, 详细设计文档

灰盒测试

- 概念: 介于黑盒与白盒之间的一种测试方法, 需要了解代码逻辑, 重点验证程序的功能
- 案例: 商城中, 用户支付时可以使用优惠券
  - 黑盒角度, 考虑各个用户, 各个商品, 各种价位, 各种金额大小的优惠券
  - 灰盒角度, 先了解业务逻辑, 再进行测试
  - # 业务逻辑
    if 商品金额 >= 优惠券金额
    　　减去即可
    if 商品金额 < 优惠券金额
    　付0元
  -

与各测试阶段的对应关系

单元 --> 白盒

集成 --> 白盒

系统 --> 黑盒/灰盒

验收 --> 黑盒

面试题

白盒测试和黑盒测试的区别?[1]

- 首先, 策略不同, 黑盒不考虑代码逻辑, 白盒需要考虑

- 其次, 阶段不同, 白盒进行较早, 单元和集成测试都属于白盒, 黑盒进行得较晚, UI/功能/易用性等都属于黑盒
- 再次, 上手难度不同, 白盒要求具备代码能力, 上手难度较大, 黑盒要求测试/用户思维, 上手难度较小
- 最后, 我想说, 黑盒和白盒不能互相替代, 都是不可或缺的

5.3 按是否运行划分

静态测试

- 概念: 指不运行被测程序本身, 检查文档或者源程序的语法, 结构, 过程等
- 测试对象
  - 文档 (需求文档, 各类设计文档)
  - 源程序
  - 数据

动态测试

- 概念: 指通过运行被测程序, 进行测试
- 测试对象
  - 源程序
  - 目标程序
  - 数据

5.4 按是否自动化划分

手工测试: 手工的方式去执行测试

自动化测试: 需要借助工具或代码去执行测试

5.5 其他

冒烟测试

针对最基本的功能或者流程进行的测试

回归测试

- 概念: 修改代码后, 重新进行测试
- 应用场景

- bug回归: 当前迭代的bug修复后, 需回归, 以前的版本中概率性出现的bug, 可能需要连续跟踪几个版本
- 旧功能回归

面试题

回归测试的重点?[3]

我做回归测试的方向主要有两个: bug回归和旧功能回归

- 如果是bug回归, 重点是修改过的功能或者模块, 还有与之关联的功能或模块
- 如果是旧功能回归, 重点是分清重要的核心功能, 以及使用频率高的功能

随机测试[了解]

随机去测试其他人没有测到的部分

探索性测试[了解]

测试设计与测试执行同时进行

6 最佳实践流程-2

测试流程

1. 需求分析, 需求评审
2. 编写测试计划
3. 编写测试用例以及发起用例评审
4. [可选]搭建测试环境, 冒烟测试
5. 系统测试, 编写测试报告
    a. 第一轮 --> 新功能的功能测试
    b. 第二轮 --> bug回归测试
    c. 第三轮 --> bug回归测试
    d. ...
    e. 最后一轮 --> 旧功能的回归测试
6. 验收测试, 编写测试报告
7. 线上, 点点点

> 测试报告一般保留上线前的那一份即可

时间规划

#以周迭代为例

需求分析 + 需求评审 -- 0.5天

测试计划 + 测试用例以及评审 -- 2.5天

系统测试 -- 1.5天

验收测试 -- 0.5天

#以两周一个迭代为例 (那么各阶段时间 * 2 即可)

面试题

你们公司的测试流程是怎样的?[3]

1. 首先是产品经理发起需求评审会议, 各角色都参会评审, 直到PRD定稿
2. 组长输出测试计划 (组长分配测试任务)
3. 测试人员根据自己的任务, 设计测试用例以及发起用例评审
4. 开发人员完成冒烟测试, 搭建测试环境并提测
5. 系统测试一般情况下大致有4轮
    a. 第一轮执行新功能的测试用例, 会发现很多bug
    b. 开发修复bug后, 第二轮进行bug回归测试
    c. 第三轮, 对修复失败的bug和新引入的bug再次进行回归测试
    d. 最后一轮, 已知bug基本清0, 对旧功能做一轮回归测试
6. 系统测试结束后, 测试人员辅助产品经理完成验收测试, 通过后, 测试人员编写测试报告, 开发做上线前的准备
7. 上线后, 测试人员去线上点点点, 验证线上有没有明显的bug

冒烟测试的作用?[2]

- 如果没有经过冒烟测试, 开发直接提测, 出现阻塞性bug, 会影响团队的进度
- 冒烟测试能确保主业务流程没问题, 也能避免返工的情况出现, 可以提高整体效率

7 测试计划和测试方案

测试计划

- 概念: 是指描述了要进行的测试活动的范围, 资源和进度的文档

- 内容
  - 测试的目标和测试范围
  - 角色分工和职责
  - 任务的安排/进度与资源分配
  - 风险评估和应急计划
  - 测试的各项标准

> - 测试计划的重点
> - 资源分配 (人, 时间, 其他测试资源)
> - 风险评估 (测试不到的部分)
> - 时间不够, 测不完
> - 技术不足, 无法进行测试
> - 场景无法模拟, 无法进行测试

测试方案

- 概念: 是从技术的角度去分析, 重点体现了测试策略与技术实现的文档
- 内容
  - 测试策略
    - UI, 功能, 易用性, 兼容, 性能, ...
  - 测试方法
    - 手工 or 自动化
    - 黑盒 or 白盒
  - 测试工具的设计和选择

测试计划和测试方案的区别

- 测试计划是**管理型**文档, 测试方案是**技术型**文档

> - 测试计划主要解决**谁来做?做什么?** 测试方案主要解决**怎么做?**

- 在**敏捷过程**中, 可以把**测试计划**看作是**测试计划+测试方案**, 并且文档也是可以省略的

面试题

有哪些常见的系统测试的策略?[1]

UI, 功能, 兼容, 易用性, 性能, 安全等等

你们公司的测试计划一般是怎么写的?[2]

- 首先是做好资源分配工作, 包括开始和结束时间, 人力方面的资料, 服务器和测试机等测试资源
- 其次是做好风险评估, 把技术和场景原因没办法测试的 或者 时间不够导致可能测不完的内容列出来, 并给出应急办法, 当然还有各阶段的通过标准
- 当然还包括测试策略的选择等等

8 测试用例

概念

指导对软件进行操作, 帮助证明软件功能或发现软件缺陷的一种说明, 最终形成文档

测试用例示例 (登录)

用例ID	模块	标题	前置条件	测试步骤	测试数据	预期结果	优先级
WMS_login_001	登录	[WMS]_登录_输入正确的用户名和密码, 登录成功	网络正常	1.打开浏览器, 输入地址 2.输入用户名和密码 3.点击登录按钮	用户名: jay 密码: 123456	登录成功, 页面自动跳转进入首页	P0
WMS_login_002	登录	[WMS]_登录_输入正确的用户名和错误的密码, 登录失败	/	1.打开浏览器, 输入地址 2.输入用户名和密码 3.点击登录按钮	用户名: jay 密码: xxxyyy	登录失败, 弹出提示"密码错误"	P1

软件测试用例包含的内容 (8大要素)

- 用例ID (用例编号) : 标识了测试用例的唯一性
- 模块
- 标题: 见名知意
- 前置条件 (本模块的前置模块, 外在网络, 服务等环境是否正常)
- 测试步骤
- 测试数据
- 预期结果 (参照需求文档)
- 优先级 (在资源有限时, 测试用例执行的先后顺序)

作用

- 降低测试人员的交接成本
- 用于评估测试工作量, 提前准备数据, 分配任务, 把控进度
- 便于回归测试

面试题

测试用例一般包含哪些内容?[3]

- 首先是ID, 模块, 标题
- 然后是前置条件, 测试步骤, 测试数据, 预期结果
- 最后还有优先级

9 测试用例设计方法

应用场景

黑盒测试的测试用例是无法穷尽的, 我们需要科学的方法帮我们把无限变为有限

常见的8个测试用例设计方法

- 等价类
- 边界值
- 判定表
- 因果图 [了解]
- 正交法 [了解]
- 场景法
- 流程图
- 错误推测法 [了解]

9.1 等价类[重点]

也叫 **等价类划分法**

概念

把程序的输入域划分成若干部分(子集), 然后选取少数代表性数据作为测试用例

> 理解什么是输入域

需求说明:

1. Logo，静态图片直接展示
2. 用户手机号码，手动填写
3. 输入获取的动态密码
4. 用户点击“获取动态密码”，如果发送成功，页面提示“已发送”，按钮文案也修改为“60秒后重新获取”，按钮不可点击，倒计时为0秒是按钮文案还原为“获取动态密码”。
5. 点击登录，后台进行判断不同角色进入不同的主页。
   目前角色有：政府机关、保险公司、第三方机构、企业四种

应用重点

- 在需求中把功能对应的**条件**找出来, 满足条件为 **有效等价类**, 不满足条件为 **无效等价类**
- 划分时, 子集互不相交, 且子集合并后是整个集合

> 例如
>
> - 数量是1到99
> - 有效等价类: 1~99
> - 无效等价类:
> - 小于1
> - 大于99

举例

需求: QQ账号输入, 6~10位自然数

1. 需求分析: 把条件找出来
   - A: 6~10 位
   - B: 自然数

2. 划分等价类

输入项	有效等价类	编号	无效等价类	编号
QQ账号	6~10位	1	小于6位	2
			大于10位	3
	自然数	4	字母	5
			汉字	6
			特殊字符	7

3.尝试设计测试用例

编号	测试数据	覆盖等价类	说明
1	66668888	1, 4	合法输入
2	12345	2, 4	非法输入
3	aaabbbcccddd	3, 5	非法输入
4	听君一席话如听一席话	1, 6	非法输入
5	1_2#3*	1, 7	非法输入

面试题

什么是有效等价类? 什么是无效等价类?[1]

- 满足需求, 满足功能具体条件的等价类是有效等价类
- 反之, 不满足需求, 不满足功能具体条件的等价类就是无效等价类

缺点

考虑不够全面, 可能产生一些遗漏, 漏测判断逻辑方面的错误

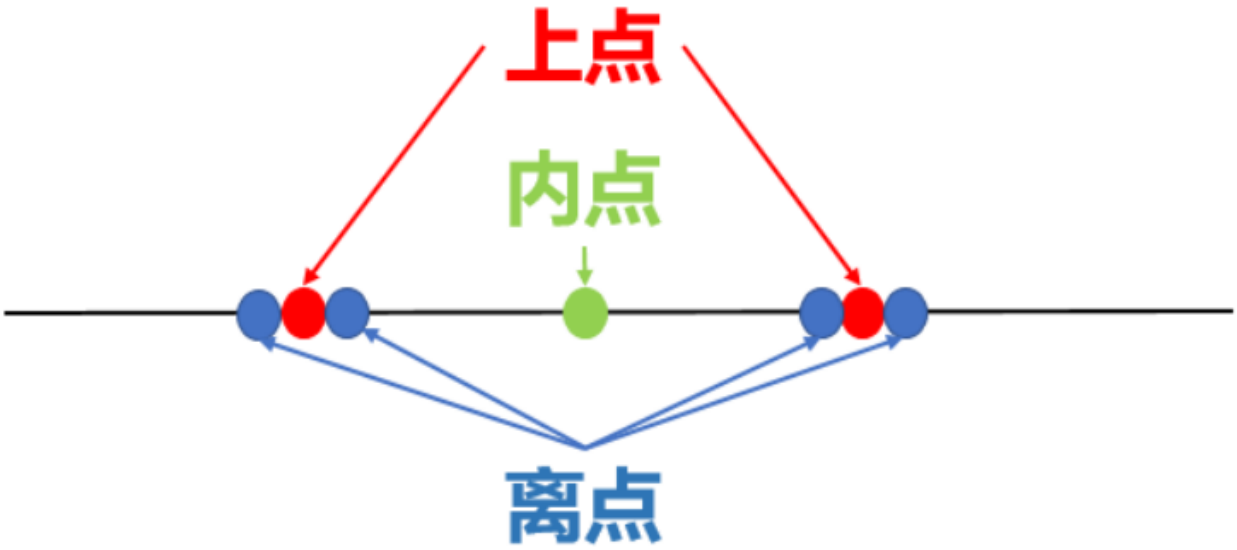
> 如, 6~10位的限制, 开发把代码写成 6<=位数<=11, 按图示设计的用例不能发现11位也能输入的bug

9.2 边界值[重点]

输入项的多数错误都发生在各种条件判断的边界上, 如果边界附近的取值没问题, 那么其他取值导致bug的概率较小

边界值

- 内点: 取值范围内的点
- 上点: 取值范围边界上的点, [6, 10] 和 (6, 10) 的上点都是6和10
- 离点: 离边界最近的点, [6, 10] 的离点是5, 11, (6, 10) 的离点是7, 9



应用重点

- 边界值法是对等价类法的重要补充
- 找出**临界点**

> 例如
>
> - 判断一个学生的成绩n是否及格 (n是0~100的整数)
> - 临界点: 0, 60, 100
> - 边界值法取值: -1, 0, 1, 59, 60, 61, 99, 100, 101
> - 上传一个20M~100M的文件, 如果文件大小的识别精度是0.1M的话
> - 临界点: 0M, 20M, 100M
> - 边界值法取值: 0, 0.1, 19.9, 20, 20.1, 99.9, 100, 100.1

举例

需求: QQ账号输入, 6~10位自然数

1. 需求分析: 涉及边界的条件只有输入的位数
2. 边界值法, 找出临界点: 0, 6, 10, 取值: -1(舍去), 0, 1, 5, 6, 7, 9, 10, 11

有效等价类	有效	编号	无效等价类	无效	编号
6~10位	6位	1.1	小于6位	空	2.1
	7位	1.2		1位	2.2
	9位	1.3		5位	2.3
	10位	1.4	大于10位	11位	3
自然数		4	字母		5
			汉字		6
			特殊字符		7

1. 尝试设计测试用例

编号	测试数据	覆盖等价类	说明
1	123456	1.1, 4	合法输入
2	1234567	1.2, 4	合法输入
3	123456789	1.3, 4	合法输入
4	1234567891	1.4, 4	合法输入
5		2.1	非法输入
6	1	2.2, 4	非法输入
7	12a21	2.3, 5	非法输入
8	aaabbbcccdd	3, 5	非法输入
9	听君一席话如听一席话	1.4, 6	非法输入
10	1_2#3*	1.1, 7	非法输入

注意

- 等价类+边界值适用于单一输入的情况
- 对于多个有逻辑关系的输入的情况不适用

9.3 判定表[重点]

判定表, 也叫决策表, 它可以把复杂的逻辑关系和多种条件组合的情况表达清楚

判断表的组成元素

判定表由条件桩, 条件项, 动作桩, 动作项组成

- 条件桩: 被测对象的所有输入条件

- 条件项: 被测对象的输入条件的取值
- 动作桩: 被测对象所有可能采取的动作/结果
- 动作项: 在各个条件项的组合下, 被测对象所采取的动作/结果

> 例如: 判断一个学生是不是三好学生 (学习, 思想品质, 身体都要好)
>
> 三个条件, 每种条件有2个取值, 则一共有2×2×2=8种组合, 每一种组合都是一条用例, 判定表的核心价值是**遍历**

举例

需求: 若用户欠费或者关机, 则不允许主被叫

1. 需求分析
    - 条件(输入)有 **欠费** 和 **关机**
    - 结果(输出)有 **允许主被叫** 和 **不允许主被叫**
2. **通过判定表法, 列表分析**

		1	**2**	**3**	**4**	
条件1	欠费?	Y	Y	N	N	
条件2	关机?	Y	N	Y	N	
结果1	允许主被叫				√	
结果2	不允许主被叫	√	√	√		

> 1. 判定表的化简

9.4 因果图

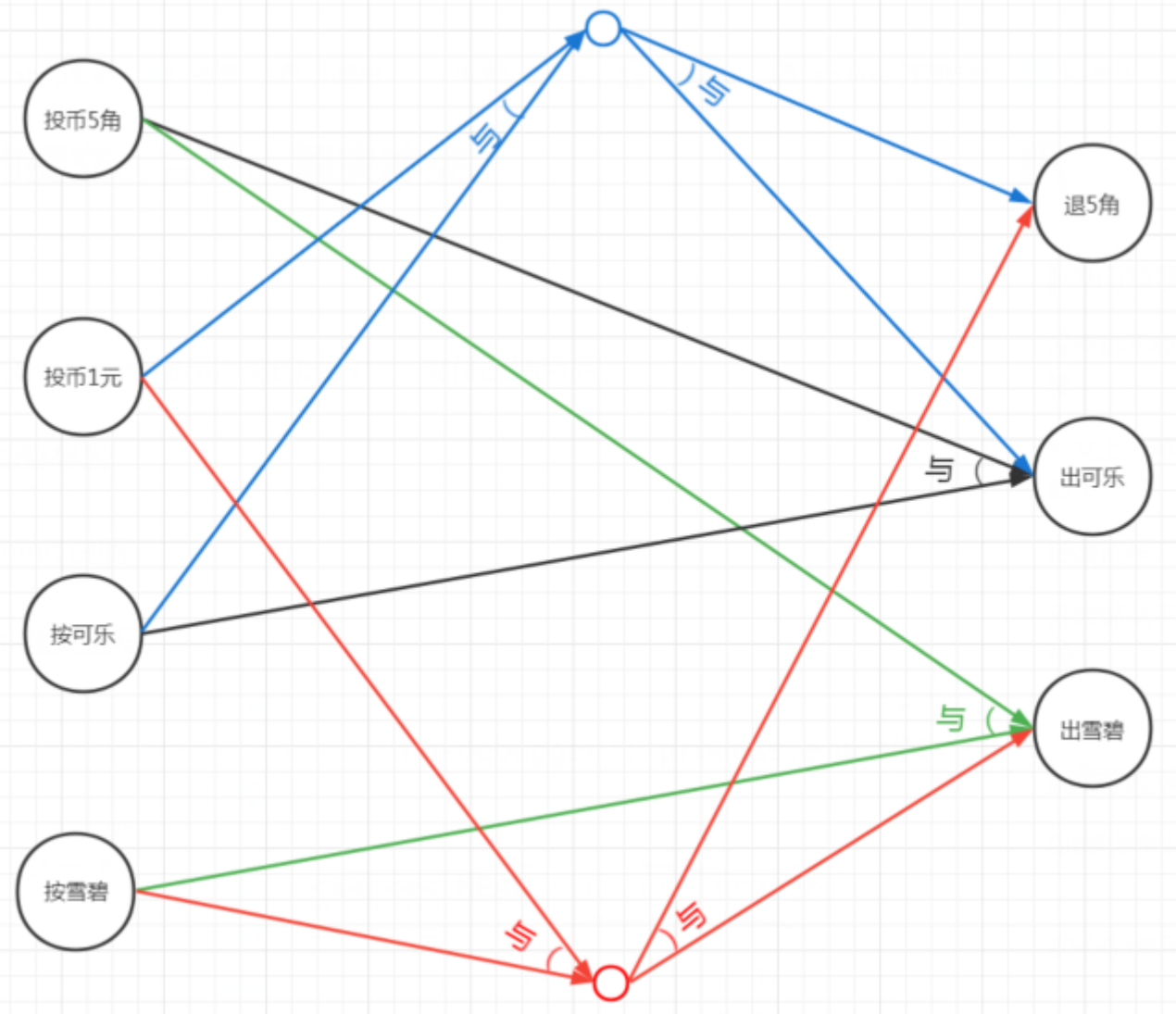
因果图是通过分析输入条件之间的关系以及输入和输出之间的关系, 再转化成判定表, 从而设计测试用例的方法

举例 (了解)

需求: 单价为5角钱的盒装饮料的自动售货机软件

若投入5角硬币，按下“可乐”/“雪碧”按钮，相应的饮料就送出来。

若投入1元硬币，送出饮料的同时退还5角硬币。



说明

直接使用判定表

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
投币5角	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
投币1元	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
按可乐	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
按雪碧	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
出可乐							1	1			1				1	1
出雪碧						1		1		1				1		1
退5角						1	1		1							1
退超过5角					1								1	1	1	

化简

- 先合并1~4不给钱的情况
- 再合并5,9,13, 给钱不买东西的情况
- 12比较特殊, 得看先按的是可乐还是雪碧
- 其他都是正常的测试用例

9.5 正交法

也叫 **正交试验分解法**

概念

是一种古希腊就有的实验设计方法, 基于数学概率学知识, 设计最经济的实验路径

> 可以简单理解为是判定表的一种简化方式

举例

需求: 假设有一个用户筛选功能, 有3个输入分别是体型、年龄段、性别,

体型有3个取值：胖、适中、瘦；

年龄段有3个取值：老人、青年、儿童；

性别有2个取值：男, 女；

列号 试验号	体型	年龄	性别
1	胖	老人	男
2	胖	青年	女
3	胖	儿童	男
4	适中	老人	女
5	适中	青年	男
6	适中	儿童	男
7	瘦	老人	男
8	瘦	青年	男
9	瘦	儿童	女

> 使用 allpairs 工具自动生成

9.6 场景法[重点]

适用范围说明

- 等价类+边界值, 适合单一输入条件的功能, 如: 输入QQ号, 输入密码, 输入身份证号, 上传文件
- 判定表, 适合有组合条件输入的功能, 如: 判断三好学生, 判断是否允许主被叫等
- 多数功能涉及业务流, 需要通过多个操作步骤来实现, 这就需要用到场景法, 如: 登录, ATM取款, 微信发红包

应用重点

需要找出**基本流**和**备选流**

- 基本流: 主流正确的操作流程 (只有一个)
- 备选流: 其他操作流程 (可以有多个)

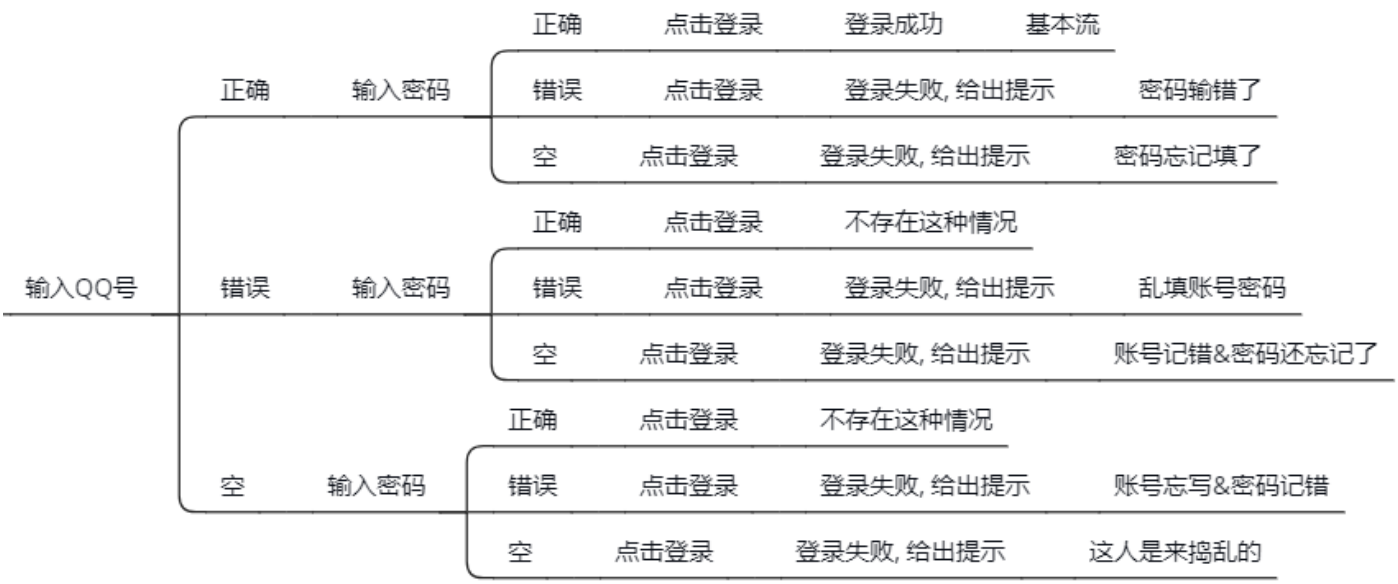
测试用例说明

- 场景1: 基本流
- 场景2: 基本流, 备选流1
- 场景3: 基本流, 备选流1, 备选流2
- 场景4: 基本流, 备选流3
- 场景5: 基本流, 备选流3, 备选流1
- 场景6: 基本流, 备选流3, 备选流1, 备选流2
- 场景7: 基本流, 备选流4
- 场景8: 基本流, 备选流3, 备选流4

举例1

需求: 测试QQ登录的功能, 用场景法设计测试用例

分析QQ登录功能的业务流所涉及的步骤: 1.输入QQ号 -> 2.输入密码 -> 3.点击登录

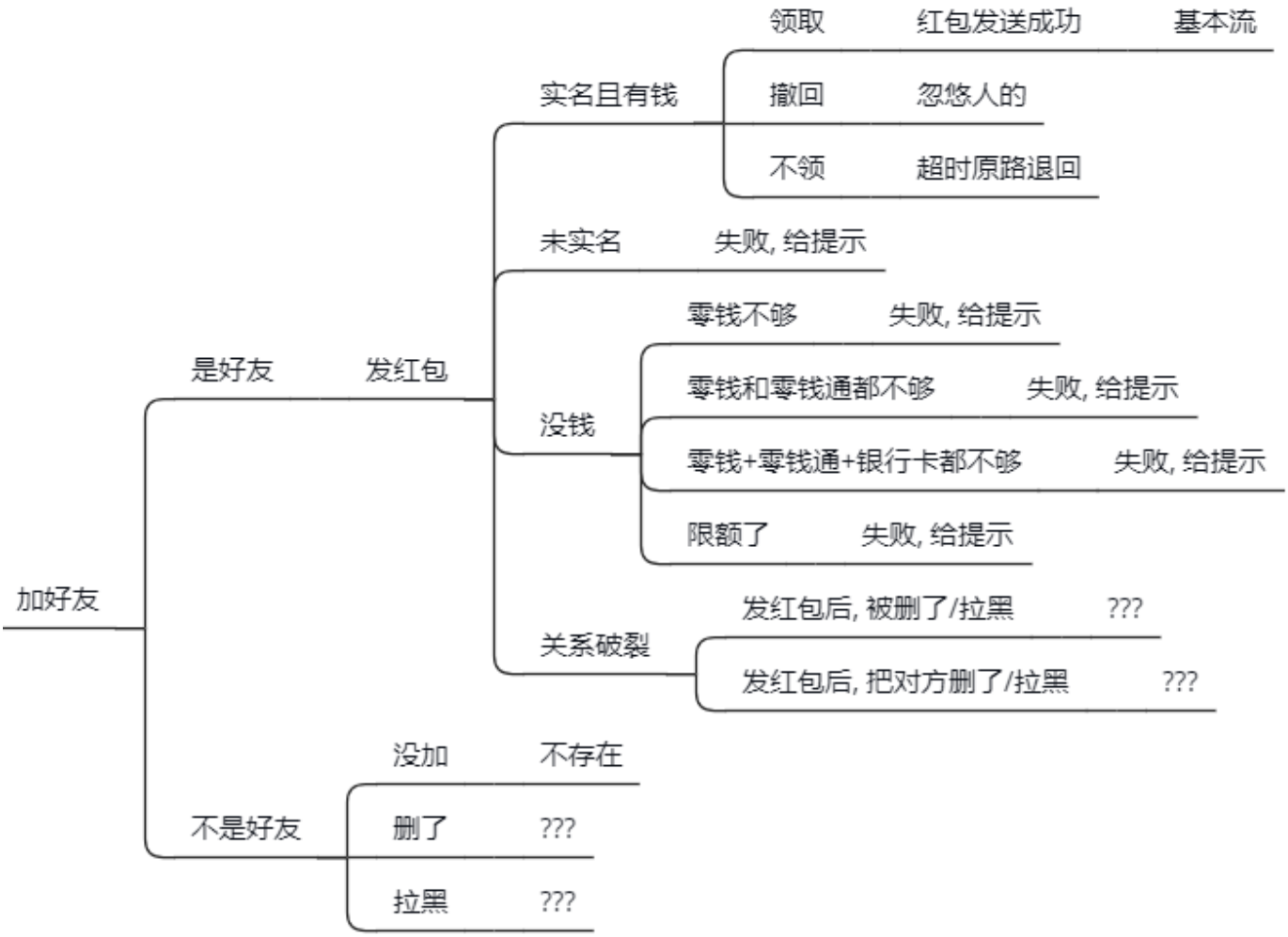


> 说明
>
> - 场景法只验证业务流程, 不验证单一功能
> - 一般先用等价类/边界值/判定表对单个功能进行验证, 通过后, 再用场景法进行流程验证

举例2

需求: 测试微信发个人红包的功能, 用场景法设计测试用例

分析微信发个人红包涉及的步骤: 1.加好友 -> 2.发红包 -> 3.对方领取



> 场景法的使用难点
>
> 备选流尽量找全, 如上图就遗漏了支付密码错误的场景

9.7 流程图法[重点]

应用场景

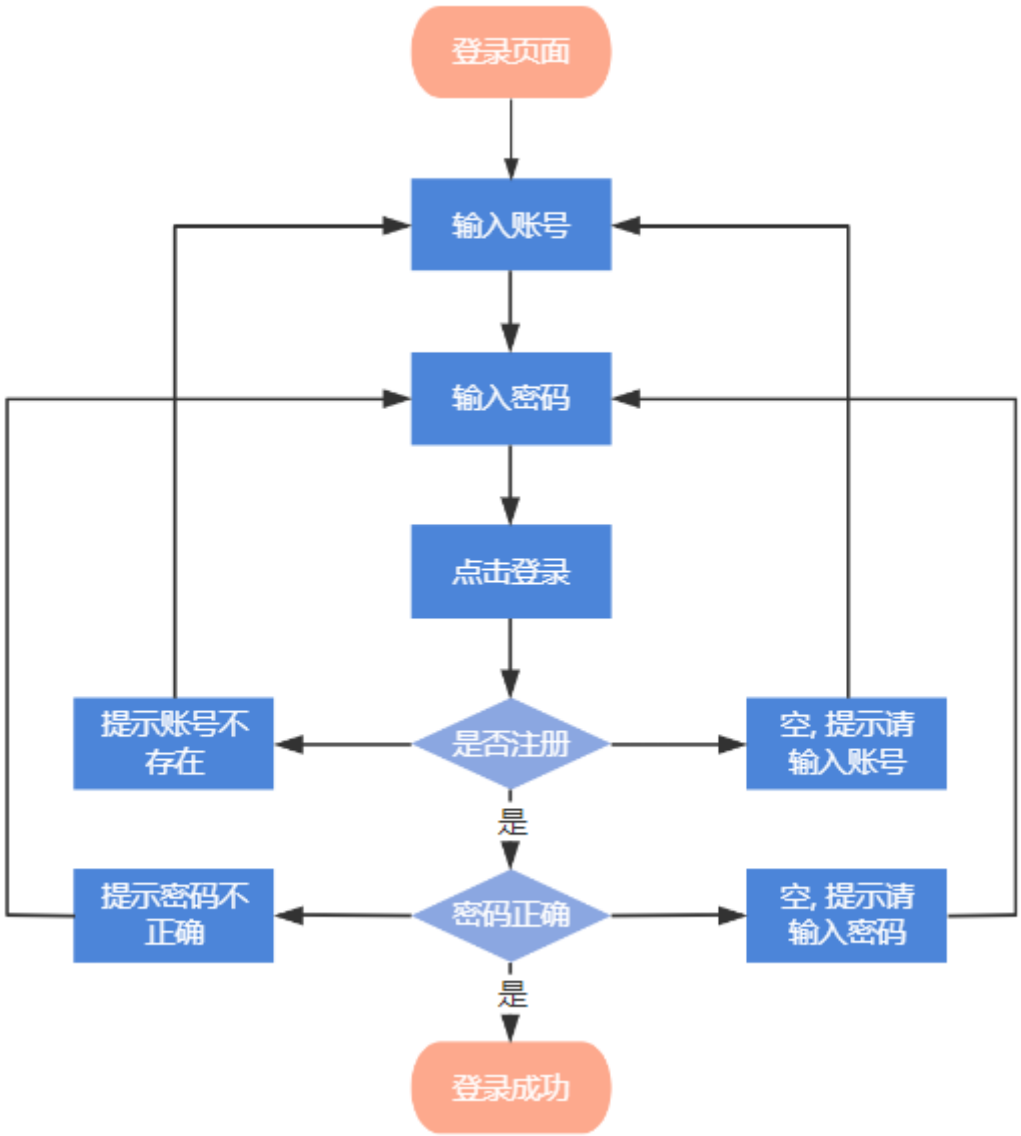
当功能的业务逻辑和实现步骤较多较复杂时, 可以借助画流程图的方式去辅助分析用户场景

画图符号规范

元素	名称	说明
	开始/结束	表示流程的开始与结束，流程图中一般只有一个开始，可以有多个结束
	流程	流程中具体的操作
	判定	不同场景输出内容不同时使用，一般应用在审核或其他系统判定中。输出的流程可以 2 至多个。
	子流程	当流程图涉及子流程过多是可用此元素来包含子流程全部内容，从而避免流程中子流程过多影响主流程的查看。
	数据	数据的输入或输出，当流程中需要用户输入或系统输出某些数据时可用此元素表示。
	文档	文档的输入或输出，当流程中需要用户输入或系统输出某些文档时可用此元素表示。
	注释	确切来说注释并不是流程图元素之一，但在流程图实际应用中注释可以对某一流程起到很好的解释作用，帮助接收方了解整个流程。 注释没有标准的表现形式，但为与流程内容进行区分一般采用填充的形式。

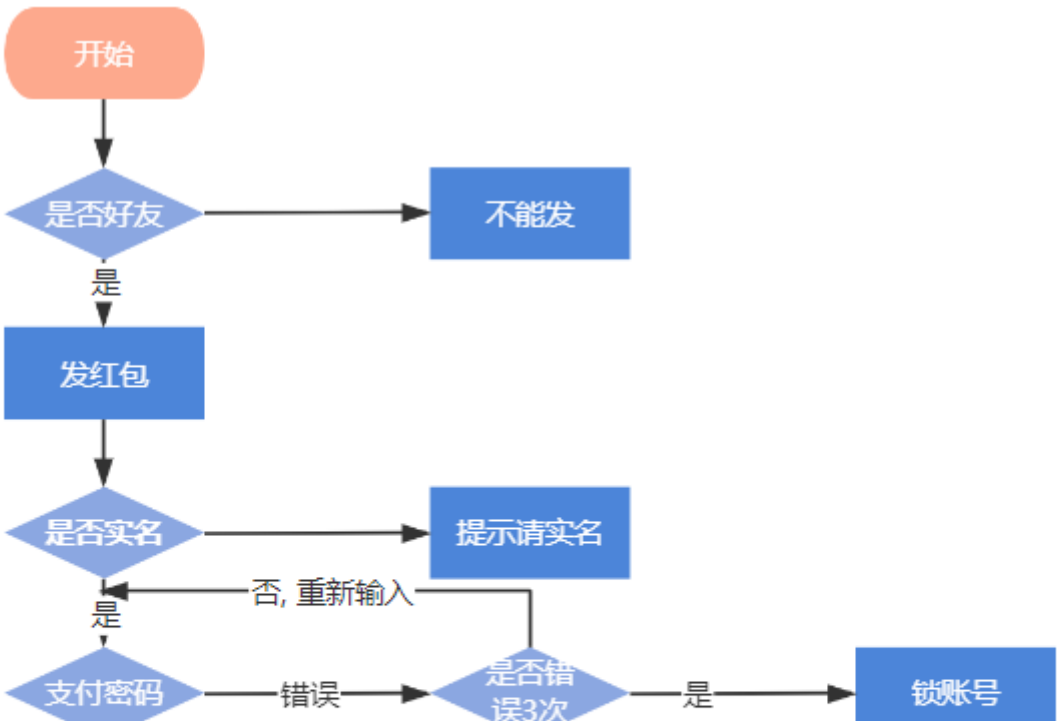
举例1

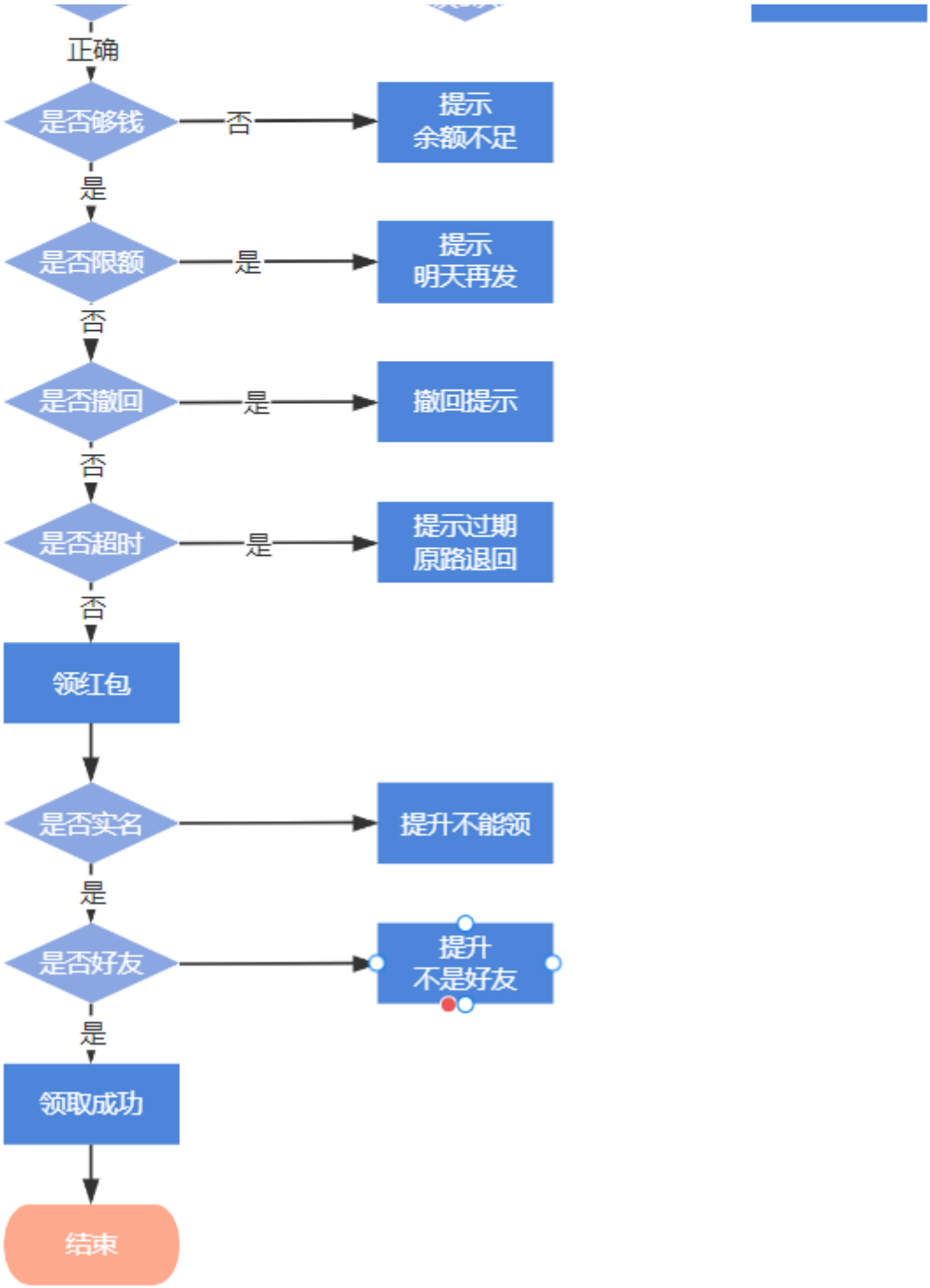
需求: QQ登录功能, 画出业务流程图



举例2

需求: 微信个人发红包功能, 画出业务流程图





> 画图工具:
>
> - visio: 属于微软的 office 系列, 但不在 office 套件中, 需单独安装
> - 在线画图工具: www.processon.com

9.8 错误推断法

应用场景

测试人员使用经验或直觉去发现程序的错误

问题: 有经验的老鸟, 是如何使用此方法的?

如, 开发人员在开发新功能时

- 基于对开发人员的了解, 感觉这个开发编程不严谨, 容易在边界判断上出错
- 旧数据的兼容问题, 如开发了手机号+验证码登录的新方式, 旧的账号密码这种关联功能容易出错
- 异常场景考虑不全面, 如空值得处理问题, 弱网问题, 性能问题等

面试题

黑盒测试的用例设计方法有哪些?[3]

- 有等价类, 边界值, 判定表, 场景法, 流程图这些都是我经常使用的
- 除此之外还有因果图, 正交法, 错误推断法等

10 软件缺陷与缺陷管理

10.1 软件缺陷概述

概念

软件缺陷就是软件的毛病, 它可能存在于UI/功能/兼容/易用性/性能等各个方面, 包括已发现的和未发现

业内对bug的通俗理解

- 需求文档**要求**的功能, 未实现
- 需求文档**虽未明确提及但应该实现**的功能, 未实现
- 需求文档**未提到**的功能, 实现了
- 软件**难以理解/不易使用/运行缓慢** 或 (从测试人员角度看) **用户会认为不好**的

> 概括的说: 实际结果与预期结果不一致

缺陷产生的原因

- 需求原因: 文档错误 / 有疏漏等
- 编码原因: 设计有误 / 编码错误等
- 其他原因: 时间紧, 沟通理解有问题, 甚至比如立项就是个错误等

描述一个缺陷的要素

- ID: 唯一性
- 模块
- 缺陷标题: 见名知意

- 严重程度
  - 严重: 主要功能不可用, crash / 闪退, 不能启动
  - 一般: 次要功能不可用, 边界 / 异常未进行处理等
  - 微小: UI问题, 易用性问题, 建议等
- 优先级
  - 高: 阻塞性问题, 影响继续测试, 需立刻修复
  - 中: 正常流程, 本次迭代上线前修复即可
  - 低: 可以延期到下个版本解决
- 复现步骤
- 预期结果
- 实际结果
- 缺陷类型: 代码错误 / 界面错误 / 兼容性错误 / 易用性错误 / 性能问题 / 安全问题
- 缺陷状态
  - 新建
  - 已指派
  - 已修复 (已解决) / 拒绝 / 延期
  - 关闭 / 重新激活

10.2 缺陷报告

概念

一个记录缺陷的文档

> **组成**
>
> 包括了缺陷描述的全部内容, 除此之外还要测试人员, 测试日期, 解决人员, 解决日期, 解决方案等

测试人员		解决人员	
测试日期		解决日期	
		解决方案	
BugID		复现步骤	
功能模块		预期结果	
标题		实际结果	
严重程度		缺陷类型	
优先级		缺陷状态	

注意事项

- 一个缺陷一个报告
- 尽量确保缺陷可以重现

功能验证表

只需要在测试用例的基础上, 加上测试结果, 测试人员, 测试日期, 解决人员, 解决日期, 解决方案等

背景: 学习视频 `24 测试理论-10-软件缺陷与缺陷管理-3-禅道介绍` **中提到缺少禅道软件的联系答疑老师**

目标: 拥有属于自己的禅道

内容: 由于是第一次接触学习环境, 需要依次学习如下内容

- 0【虚拟机】从零开始拥有流畅的学习环境
  - 虚拟机软件VMware_Workstation_Pro安装
    - [可选] 虚拟机软件VMware_Workstation_Pro卸载
  - 虚拟机1: 禅道+api1.0+api_server
    - 虚拟机1的应用1: 禅道的启动和访问

10.3 禅道介绍

背景

- 以前那种缺陷报告的形式不便于管理, 所以开发了各种管理工具

- 专门用于bug管理的有: bugfree/bugzilla/mantis等等
- 专门用于测试用例管理的有: testlink等
- 用于项目综合管理的, 包括项目/模块/文档/测试(用例+bug)等的工具有: 禅道/Jira等
  - 其中禅道 (zentao) 是国产软件
  - 洋气一点的公司选择用Jira

禅道的工作流

测试用例管理介绍

- 步骤
  a. 编写用例: 测试 -> 用例 -> +建用例
  b. 用例评审: 导出 -> 线下进行评审
  c. 执行测试用例
- 说明
- 基于互联网产品的迭代节奏, 一般更倾向于直接用excel来管理测试用例, 或者思维导图管理测试点(而不写用例)

10.4 用禅道管理bug

bug的生命周期

1. 发现bug
2. 禅道上提交bug: 测试 -> Bug -> +提Bug
3. 禅道上指派bug, 其实就是通过bug定位, 指出让谁去解决这个bug
4. 线下修复这个bug, 在禅道上标记"已解决", 并再次指派由哪个测试进行验证
5. bug回归测试, 如果验证通过则在禅道上"关闭"这个bug

> 1. 如果验证不通过可在bug详情右上角重新"激活"这个bug

面试题

bug的生命周期?[2]

发现 -> 确认 -> 提交 -> 指派 -> 修复 -> 验证 -> 关闭 或者 重新激活再次进行指派

你们公司的缺陷管理流程是怎样的?[2]

- 首先是发现并定位这个bug
- 然后是在禅道上提交并指派给对应的开发
- 开发修复后会标记"已解决"并指派给回测试
- 最后是bug回归验证, 验证通过就关闭这个bug, 如果验证不通过就重新激活

bug的描述信息包含哪些内容? 缺陷报告包含哪些信息?[2]

- 主要有: ID, 模块, 标题, 严重程序, 优先级, 复现步骤, 预期结果, 实际结果
- 还有: bug类型, bug状态, 以及附近截图等

bug的严重程度和优先级的区别 (或者问怎么判断)?[2]

- 严重程度
  - 高, 是主要功能不可用, crash, 闪退等问题
  - 中, 次要功能不可用, 边界/异常未处理等问题
  - 低, 易用性问题, UI问题, 小的性能问题等
- 优先级
  - 高, 阻塞性问题, 影响测试了, 例如不能登录
  - 中, 正常解决, 本次迭代上线前修复即可
  - 低, 可以延期到下个版本解决

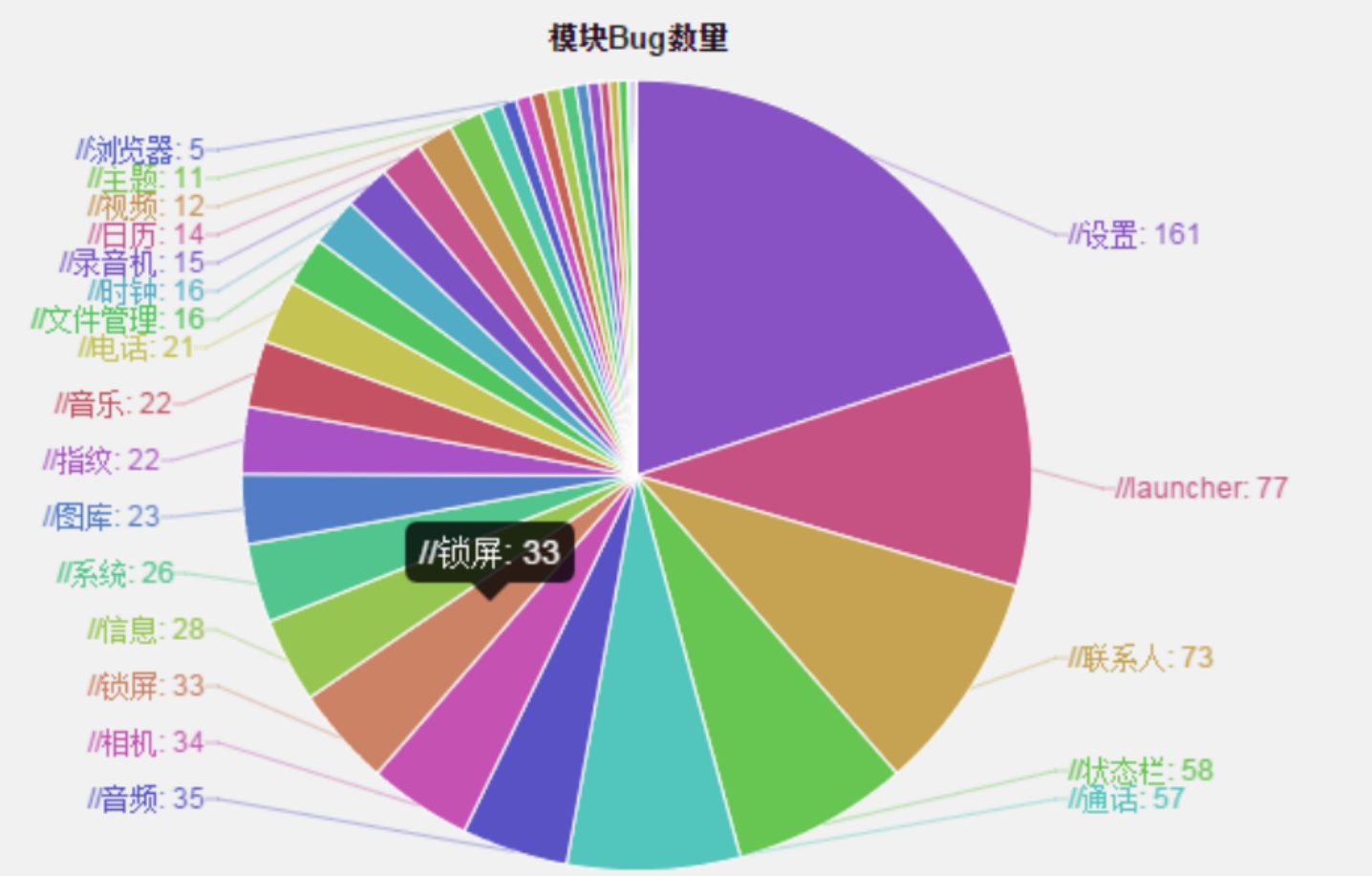
不过对于我们公司来说, 我们的要求是bug(当天)清0, 所以严重程序和优先级默认即可

缺陷清单 (BugList)

bug主页右上角的"导出"功能, 可以导出 BugList, 作为测试报告的一部分

缺陷统计

- 示例: bug主页右上角"报表"功能



- 作用
  - 了解隐患, 方便项目相关负责人知悉当前测试版本的整体质量
  - 绩效考核
    - 测试人员提交bug的量化指标
    - 开发人员的bug分布

11 测试报告

什么是测试报告

测试活动的总结性文档, 标志测试活动的结束

测试报告都包含什么内容

- 测试工作的经过和**结果**
- 缺陷的汇总和分析
- 风险评估
- 测试工作的总结和改进

> 模板: https://www.cnblogs.com/tv151579/p/8250157.html

面试题

你们公司的测试报告包括什么内容?[2]

- 首先是关键信息, 如: 项目名, 版本号, 测试结果, 完成时间等
- 然后是带上缺陷清单和一些统计信息, 比如测试用例总数, 用例执行率, bug总数, 遗留bug数量等
- 之后是风险评估, 评估未能执行的用例和遗留bug可能带来的风险, 也包含其他业务风险等
- 最后就是工作总结, 提出具体的工作改进建议

你们公司测试退出的标准是什么? 项目交付的标准是什么?[2]

- 测试用例已经通过评审
- 测试用例执行率是100%
- bug清0了
- 有经过验收测试, 并且验收通过

12 网络基础简介

12.1 网络通信模型

两种通信模型

- **OSI模型**是由国际化标准组织制定的计算机网络通信的参考模型, 常叫**OSI七层模型**
- **TCP/IP模型**是在实际的互联网发展中逐渐演化而来的模型, 常叫**TCP/IP四层模型**

图示

<table>
<tr><th>OSI七层模型</th><th>作用</th><th>TCP/IP四层模型</th><th>代表协议</th></tr>
<tr><td>应用层</td><td>实现服务, 开放端口</td><td rowspan="3">应用层</td><td rowspan="3">HTTP, FTP, SSH, DNS等</td></tr>
<tr><td>表示层</td><td>数据处理(编解码/加解密)</td></tr>
<tr><td>会话层</td><td>会话管理和流量控制</td></tr>
<tr><td>传输层</td><td>保证数据传输, 但不传</td><td>传输层</td><td>TCP, UDP</td></tr>
<tr><td>网络层</td><td>路由, 实际传数据</td><td>网络层</td><td>IP</td></tr>
<tr><td>数据链路层</td><td>把比特流变成帧, 控制可靠性</td><td rowspan="2">网络接口层</td><td rowspan="2">ARP/RARP
USB/蓝牙/WIFI/以太网</td></tr>
<tr><td>物理层</td><td>物理媒介, 传输比特流</td></tr>
</table>

用户				原始数据							
应用层			HTTP	原始数据							
传输层			TCP头	数据块1					TCP头	数据块2	
网络层		IP头	TCP头	数据块1				IP头	TCP头	数据块2	
网络接口层	帧头	IP头	TCP头	数据块1	帧尾		帧头	IP头	TCP头	数据块2	帧尾

> **说明**
>
> - 用户: 只需要思考发什么东西(数据)给别人
> - 应用层: 像是快递上门把件打包, 贴个标签取走了
> - 传输层: 发现包裹太大, 得拆分, 还得加个顺序, 不然收货的就组装不起来了
> - 网络层: 通过路由, 找收件人, 开始发货
> - 网络接口层: 网络数据太杂乱, 带上帧头和帧尾, 表示这段数据才是真正的货

面试题

简述OSI七层模型和TCP/IP四层模型的结构, 以及各层的重要协议?[3]

- OSI七层模型分为应用层, 表示层, 会话层, 传输层, 网络层, 数据链路层 和 物理层
- TCP/IP四层模型分为应用层, 传输层, 网络层, 网络接口层
  - 应用层重要协议有 http, https, ftp协议等
  - 传输层重要协议有TCP, UDP
  - 网络层有IP协议
  - 网络接口层的协议有 ARP/USB/蓝牙/WIFI等协议

12.2 重要协议

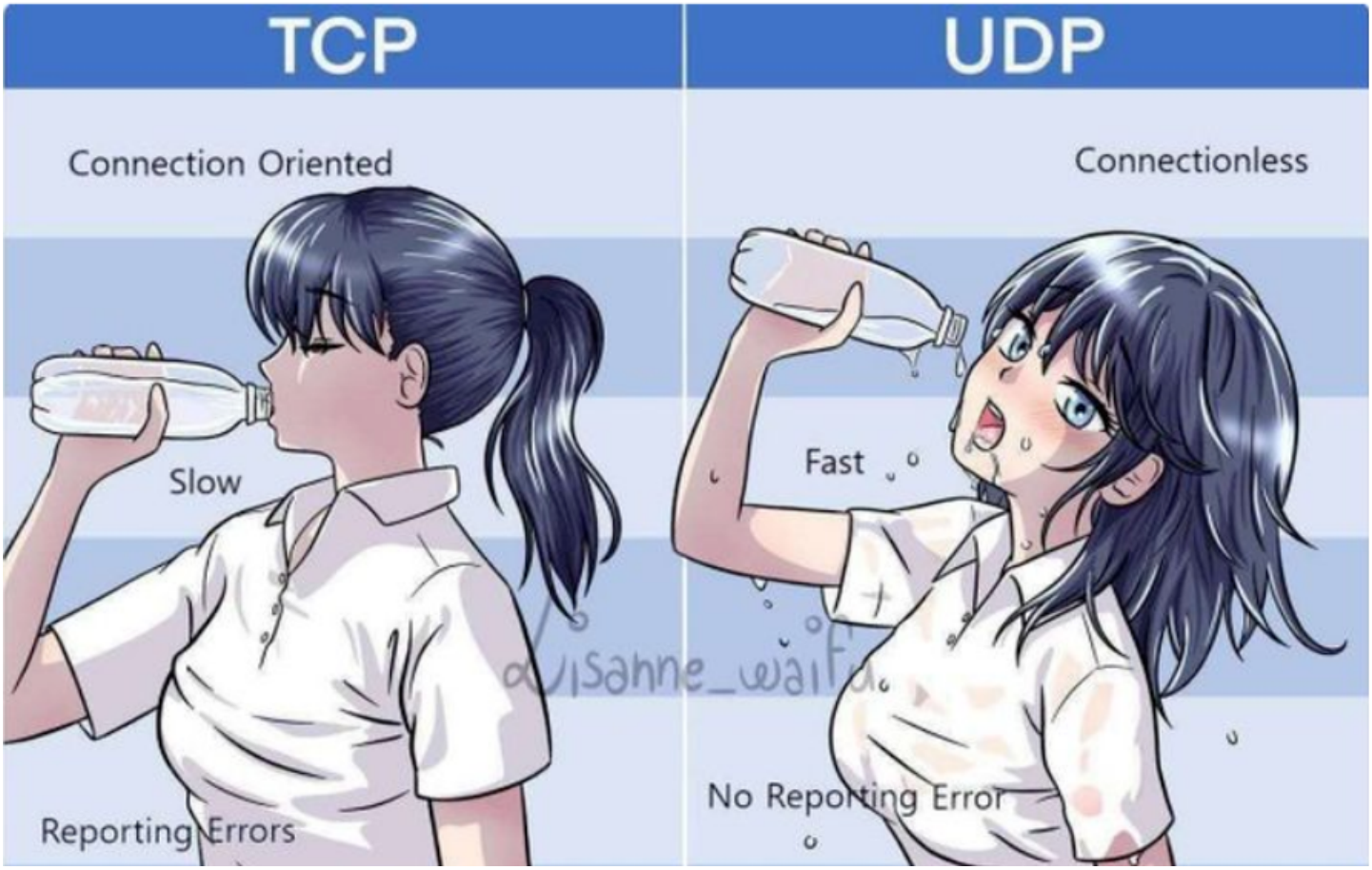
TCP协议特点

- 面向连接, 先建立连接, 再传输数据
- 可靠, 它会基于内部各种机制保证数据到达目的地

UDP协议特点

- 无连接, 只需要知道对方的IP和端口号就可以发送数据了, 无需建立连接
- 不可靠, 没有一系列机制来应对丢包问题

神图



面试题

TCP和UDP的区别?[3]

- TCP是基于连接, UDP是无连接的
- TCP对系统资源的消耗比UDP多
- TCP保证传输数据的到达以及数据包的顺序, UDP可能会丢包或者乱序
- TCP比UDP慢
- 常规的业务用TCP, UDP主要传输音频, 游戏中实时性要求较高的业务也用UDP, 比如王者5v5

HTTP协议特点

> HTTP协议, 也叫超文本传输协议 (超文本: 图片, 视频, 音频)

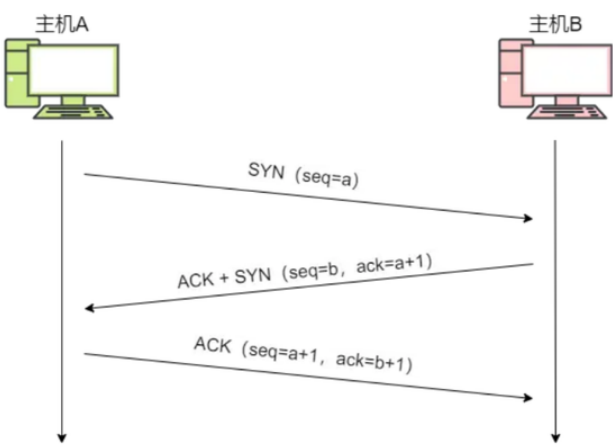
- HTTP连接是一种短连接, 它通过TCP和服务器建立连接, 数据传输结束后, 立即将TCP连接断开
- 同时HTTP连接是无状态的连接, 指每次向服务器发起请求时, 每次都要新建一个连接

面试题

HTTP和TCP之间的关系?[1]

- 二者属于不同通信层, HTTP在应用层, TCP在传输层
- HTTP是基于TCP来实现的 (数据传输的通道是由TCP打开的)
- TCP是长连接, 会消耗服务器资源
- HTTP是短连接, 每次HTTP请求结束后, 客户端会主动断开

TCP (HTTP) 建立连接的过程 (三次握手)



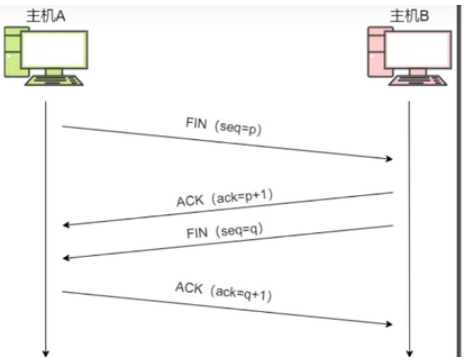
三次握手

主机 A 想向主机 B 发送数据，TCP 模块通过**三次握手**建立连接 TCP 会话。

三次握手，是指 TCP 会话建立过程中共交换了 3 个 TCP 控制段，它们分布是 SYN 段、SYN + ACK 段、ACK 段。详细过程如下：

1. 发送端主机 A 向接收端主机 B 发出 SYN 段，表示发起建立连接请求，同时把自己的状态告诉主机 B 。将段的序列号设为 a ，SYN 置位，表示 SYN 管理段。
2. 主机 B 收到连接请求后，回应 SYN + ACK 段，将序列号设为 b ，确认号设为 a + 1 ，同时将 SYN 和 ACK 置位。
3. 主机 A 收到主机 B 的连接确认后，发送 ACK 段再次进行确认，确认会话的建立，将 ACK 置位。主机 A 收到确认号是 a + 1 、序列号是 b 的段后，发送序列号为 a + 1 、确认号为 b + 1 的段进行确认。
4. 主机 B 收到确认报文后，连接建立。双方可以开始传输数据。

> **说明**
>
> - A -> B, 真漂亮, 处不?
> - A <- B, 你也很帅, 处!
> - A -> B, OK, 咱们去约会!
> - 数据传输过程, 相当于处对象的过程



四次挥手

当 TCP 数据段的传输结束时，双方都需要发送 FIN 段和 ACK 段来终止 TCP 会话。这个方式叫做**四次挥手**，详细过程如下：

1. 主机 A 想要终止连接，发送序列号为 p 的段，FIN 置位，表示 FIN 管理段。
2. 主机 B 收到主机 A 发送的 FIN 段后，发送 ACK 段，确认号为 p + 1 ，同时关闭连接。
3. 主机 B 发送序列号为 q的段，FIN 置位，通知连接关闭。
4. 主机 A 收到主机 B 发送的 FIN 段后，发送 ACK 段，确认号为 q + 1 ，同时关闭连接。TCP 连接至此结束。

> **说明**
>
> - A -> B, 腻了, 分了吧?
> - A <- B, 分就分!
> - A <-B, 早就看你不顺眼了!
> - A -> B, 好!

面试题

HTTP (TCP) 三次握手建立连接和四次挥手断开连接的过程是怎样的?[1]

- 三次握手
  a. 客户端向服务器发起建立连接的请求
  b. 服务器应答确认信息
  c. 客户端再次确认
  d. 完成三次握手后, 就可以在连接通道传输数据了
- 四次挥手
  a. 客户端向服务器发起结束对话的请求
  b. 服务器应答确认信息
  c. 服务器应答结束对话
  d. 客户端发送确认信息后, 断开连接

12.3 URL和DNS

URL

- 概念, Uniform Resource Locator, 统一资源定位符
- 格式, 协议://IP地址:端口号/资源路径
  - 如, http://127.0.0.1:1234/hello?a=1
  - 如, http://192.168.0.105:1234/hello?a=1
  - 如, http://localhost:1234/hello?a=1
  - 如, http://localhost:1234/love
  - 如, https://www.baidu.com/s?wd=12306

DNS

- 概念, Domain Name System, 域名系统
- 作用, 将域名和IP关联起来

面试题

描述一次HTTP请求的过程? 当我们在浏览器输入了一个url后会发生什么?[2]

1. 浏览器客户端输入url, 回车后

2. 通过DNS解析, 得到服务器的IP地址
3. 根据http协议, 通过三次握手建立连接
4. 发送http请求, 服务器处理请求并返回响应报文, 浏览器解析渲染页面
5. 四次挥手断开连接, 请求结束

12.4 浏览器抓包

什么是抓包

就是抓取网络传输的请求和响应的数据包的过程

> 请求, request, 客户端发给服务器的数据
>
> 响应, response, 服务器返回给客户端的数据

为什么要抓包

- 对测试人员来说最主要的作用是定位bug
- 抓包除了能了解传输的数据之外, 还能模拟重发, 修改数据包等

web程序抓包过程

1. 打开浏览器, 进入web页面
2. 打开浏览器的 F12 (开发者工具)
3. 操作你想要抓包的功能
4. 在开发者工具的"Network"(网络) 标签下即可看到抓取结果

抓包结果查看

请求行	**URL, 请求方法**	**响应行**	**http协议的版本号, 状态码**			
请求头	**浏览器信息, 支持语言等**	**响应头**	**web服务器信息, 响应内容的类型, 响应内容的长度**			
请求体	**请求数据**	**响应体**	**响应数据**			

> 以chrom浏览器为例
>
> - 请求行和响应行的内容在 General 中, 分别是
> - Request URL 请求的URL
> - Request Method 请求方法
> - Status Code 状态码
> - 请求头的信息在 Request Headers 中, 如
> - User-Agent 浏览器信息
> - Content-Type 数据类型
> - Accept-Language 接收的语言
> - 响应头的信息在 Response Headers 中, 如
> - Server 服务器信息
> - 请求体信息在 Payload 中
> - 响应体的信息在 Response 中

面试题

怎么通过抓包来定位bug属于前端还是后端?[3]

- 抓包是为了拿到请求和响应数据
- 如果请求有问题, 就是前端bug, 主要看请求方式, URL, 请求参数等
- 如果响应有问题, 就是后端问题, 主要看响应的状态码和响应体数据

13 如何表达一个功能怎么测

13.1 总体介绍

表达原则

- 全面
- 除了描述功能如何测试之外, 还要加上常用的非功能测试的思考, 如: UI, 兼容, 易用性, 性能, 安全

表达顺序

1. UI 和 功能
2. 兼容
3. 易用性
4. 性能

5. 安全

关注点

- UI, 遍历每个页面, 看页面元素是否美观, 布局是否合理, 字体字号是否统一, 是否支持更换主题或皮肤等
- 功能, 需求中描述的功能是否实现
- 兼容
  - 遍历主流的操作系统, 浏览器, 分辨率, 检查软件功能是否可用, UI是否错乱/变形
  - 重点考虑弱网, 软件是否可用, 如不可用是否给出友好提示
- 易用性, 软件是否符合用户的操作习惯, 回车事件是否支持, 所需用户操作是否复杂
- 性能, 页面响应快不快, 是否满足3-5-8原则

> - 这只是性能的其中一个方面, 后续有专门的性能测试阶段

- 安全, 数据是否安全, 业务是否安全

> - 安全测试涉及面很广, 如: 代码安全, 业务安全, 数据安全, 服务器安全等多个方面, 我们现阶段不要深究
> - 数据安全
> - 数据展示的安全性, 敏感信息是否遮挡, 是否不支持复制
> - 数据传输过程中的安全性, 是否加密
> - 数据存储的安全性, 敏感信息在数据库内是否加密
> - 业务安全
> - 密码错误次数过多是否锁账号
> - 用户会话的过期时间是否合理
> - 是否有防作弊, 防刷策略
> - 下单过程中是否支持修改订单金额
> - ...

13.2 表达练习

重难点和技巧

- 当问到一个功能你会如何测试时, 重难点都是这个功能本身的测试
- 技巧上, 需要我们在脑海中列出 **功能 -> 步骤 -> 条件**

> - 如, 问到登录功能上怎么测, 分析过程如下:

> a. 输入用户名
> - 渠道 (方式) 问题
> - 长度限制
> - 类型限制
> - 是否必填
> - 是否唯一
> - 业务问题
> b. 输入密码
> c. 点击登录

面试题

登录怎么测?[3]

- UI上, 我主要是检查登录页面, 看页面元素是否美观, 布局是否合理, 字体字号是否统一
- 功能
  - 遍历各种登录方式
  - 用户名和密码符合长度限制和不符合的时候分别测试是否能登录成功, 失败有没有提示
  - 用户名和密码符合类型限制和不符合的时候分别测试是否能登录成功, 失败有没有提示
  - 用户名和密码为空是否能登录成功, 失败有没有提示
  - 重复使用同一个用户名登录时, 会不会成功, 会不会顶号, 失败会不会提示
  - 密码错误次数过多会不会锁定账号, 会不会提示找回密码方式等
- 兼容
  - 遍历主流的操作系统, 浏览器, 分辨率, 检查软件功能是否可用, UI是否错乱/变形
  - 重点考虑弱网, 登录是否可用, 如不可用是否给出友好提示
- 易用性, 看是否支持回车提交登录操作
- 性能, 看登录结果的响应时间是否满足3-5-8原则, 再看看大量用户登录时的情况
- 安全
  - 登录密码会不会遮挡, 能不能复制, 再次登录会不会默认保存上次的账号和密码
  - 账号和密码提交时, 会不会在地址栏明文传输
  - 登录冲突时会不会被挤下线
  - 自动登录的时间限制是否合理
  - 登录有没有设备和环境检查等

支付功能怎么测?[3]

- UI上, 主要检查支付相关页面, 看页面元素是否美观, 布局是否合理, 字体字号是否统一
- 功能 (输入金额 -> 选择渠道 -> 输入密码)
  - 金额类型, 检查金额是否支持数字且支持小数点, 不支持其他类型
  - 金额大小, 检查金额的边界和无意义的值, 比如 -1, 0, 0.01, 大额边界等
  - 遍历各种支付渠道, 如微信/支付宝/银联等, 同事要主要各渠道支付上限和余额不足的情况
  - 余额不足时, 充值后是否可以继续支付
  - 遍历各种密码验证方式, 如密码/手势/指纹/人脸/免密等
  - 密码错误后是否支持重新输入
- 兼容
  - 遍历主流的操作系统, 浏览器, 分辨率, 检查软件功能是否可用, UI是否错乱/变形
  - 重点考虑弱网, 支付是否可用, 如不可用是否给出友好提示
  - 弱网下, 连续点击支付, 是否会要求支付多次
  - 断网后, 支付是否可用
- 易用性, 是否符合用户操作习惯
- 性能, 看支付的响应时间是否满足3-5-8原则, 再看看大量用户同时支付时的情况
- 安全
  - 支付密码会不会遮挡, 能不能复制
  - 支付的订单和密码, 会不会在地址栏明文传输
  - 支付失败如何继续支付或退单
  - 退款如何处理

如何测试一个水杯?[3] (如何测试一支笔, 如何测试一部电梯)

- UI
  - 外观风格是否统一
  - 颜色搭配是否合理
  - 外观大小是否适中
  - 是否有图案
- 功能
  - 是否能正常装水, 有没有刻度现实, 什么材质

    - 是否能正常喝水
    - 是否有盖子, 能不能盖住, 盖紧后是否会漏水
    - 是否有保温功能
- 兼容, 是否能装其他液体, 如: 可乐/果汁/酒精/汽油等, 以及不同温度的液体
- 易用性
    - 装水是否方便
    - 喝水是否方便
    - 携带是否方便
    - 拿起放下是否方便, 是否防滑
    - 装有低温或高温液体时, 是否会让手感到不适
- 性能
    - 装满水会不会漏
    - 可以保温多久
    - 有没有最大使用次数
    - 摔过后还能不能正常使用
    - 能装多少水
- 安全
    - 装热水会不会烫手
    - 长时间或极端环境放置, 会不会产生有毒物质
    - 能不能放进微波炉
    - 损坏后, 是否会伤人